

☐ SOFTWARE REQUIREMENTS SPECIFICATION (SRS) CM Chat App — 5 Din Ka Complete Plan

2 Developers: Frontend + Backend

Project Name: CM Chat App

Figma Design: CM Chat App (figmamarket.com) (https://www.figma.com/design/0ow1aTjRRqjoz2PjPeAn1f/CM-Chat-App--figmamarket.com-?node-id=2478-10304)

Document Version: 1.0

Date: 23 September 2026

Total Duration: 5 Din (Working Days)

Team Size: 2 Banda (Frontend Developer + Backend Developer)

☐ Table of Contents

1. Introduction
2. Project Overview
3. Figma Design Analysis
4. Tech Stack
5. Functional Requirements
6. Non-Functional Requirements
7. Database Design
8. API Endpoints Contract
9. WebSocket Events
10. 5-Din Ka Day-by-Day Plan
11. Frontend-Backend Coordination Plan

- 12. Frontend-Backend Connection Guide
 - 13. Testing Checklist
 - 14. Risk Analysis
-

1. Introduction

1.1 Maqsad (Purpose)

Yeh SRS document **CM Chat App** ke tamam requirements ko define karta hai. Yeh document **2 developers** (ek Frontend, ek Backend) ke liye likha gaya hai taake dono ko pata ho ke:

- Kya banana hai
- Kab banana hai
- Kaise connect karna hai
- Kaun kya karega

1.2 Scope

CM Chat App ek **real-time messaging application** hai jis mein:

- User registration aur authentication
- One-to-one private chat
- Group chat
- Media sharing (photos, files)
- Voice/Video call UI
- User profile management
- Status/Stories feature
- Real-time notifications

1.3 Definitions

Term	Matlab
FE	Frontend Developer
BE	Backend Developer
API	Application Programming Interface — Frontend aur Backend ke darmiyan rasta
WS	WebSocket — Real-time communication ke liye
JWT	JSON Web Token — Login ke baad user ko verify karne ka tareeqa
CRUD	Create, Read, Update, Delete — Basic operations

2. Project Overview

2.1 Design Reference

- Figma Design: CM Chat App (figmamarket.com)
- Design Node: 2478-10304
- Created: January 22, 2026
- Link: <https://www.figma.com/design/0ow1aTjRRqj0z2PjPeAn1f/>

2.2 Application Type

- **Platform:** Web Application (Responsive — Desktop + Mobile)
- **Type:** Real-time Chat Messaging App
- **Architecture:** Client-Server with WebSocket

2.3 Team Structure

- Developer 1: FRONTEND (FE)
 - ├─ UI screens banana (Figma se code)
 - ├─ State management
 - ├─ API integration
 - ├─ WebSocket client setup
 - └─ Responsive design
- Developer 2: BACKEND (BE)
 - ├─ Database design + setup
 - ├─ REST APIs banana
 - ├─ Authentication system
 - ├─ WebSocket server setup
 - ├─ File upload system
 - └─ Deployment

3. Figma Design Analysis

3.1 CM Chat App — Screens List (Design Se)

Figma design ke mutabiq yeh screens banana honghi:

- CM Chat App Screens
 - |
 - └─ □ Authentication Screens
 - | └─ Splash / Welcome Screen
 - | └─ Login Screen (Email/Phone + Password)
 - | └─ Register / Sign Up Screen
 - | └─ Forgot Password Screen
 - | └─ OTP Verification Screen
 - |
 - └─ □ Main Screens
 - | └─ Chat List Screen (Home – sab conversations)
 - | └─ Chat Detail Screen (individual chat – messages)
 - | └─ Group Chat Screen
 - | └─ Contact List Screen
 - | └─ Search Screen (chats/contacts search)
 - |
 - └─ □ Profile Screens
 - | └─ My Profile Screen
 - | └─ Edit Profile Screen
 - | └─ Other User Profile Screen
 - | └─ Settings Screen
 - |
 - └─ □ Call Screens
 - | └─ Call History Screen
 - | └─ Voice Call Screen (UI only)
 - | └─ Video Call Screen (UI only)
 - |
 - └─ □ Status/Stories Screens
 - | └─ Status List Screen
 - | └─ Status Viewer Screen
 - |
 - └─ ⚙ Other Screens
 - | └─ Notification Screen
 - | └─ New Chat / New Group Screen
 - | └─ About / Help Screen

3.2 Design Color Palette (Figma Se)

Color	Hex Code	Kahan Use Hoga
Primary Blue	#2D6CDF	Buttons, active states, links
Dark Background	#1B202D	App background (dark theme)
Card Background	#232A3B	Chat cards, panels
White	#FFFFFF	Text, icons on dark bg

Color	Hex Code	Kahan Use Hoga
Light Gray	#8E99A4	Secondary text, timestamps
Green Online	#4CAF50	Online status dot
Sent Bubble	#2D6CDF	User ke bheje messages
Received Bubble	#2A3142	Doosre ke messages
Red/Error	#FF4757	Errors, missed calls
Accent Green	#00D68F	Success states

3.3 Typography

Element	Font	Size
Headers	SF Pro Display / Inter Bold	20-28px
Chat Name	Inter SemiBold	16px
Message Text	Inter Regular	14px
Timestamp	Inter Regular	11px
Button Text	Inter SemiBold	14-16px

4. Tech Stack

4.1 Frontend (Developer 1 Ka Stack)

Technology	Kaam
React.js 18	UI components banana
Next.js 14	Routing, SSR, App Router
Tailwind CSS	Styling (Figma design match karna)
Socket.IO Client	Real-time messaging
Axios	API calls karna
React Hook Form	Form handling (login, register, profile)
Zustand ya Redux Toolkit	State management
Lucide React	Icons
date-fns	Date/time formatting
react-hot-toast	Notifications/toasts

4.2 Backend (Developer 2 Ka Stack)

Technology	Kaam
Node.js	Server runtime
Express.js	REST API framework
Socket.IO	Real-time WebSocket server
MongoDB + Mongoose	Database (NoSQL)
JWT (jsonwebtoken)	Authentication tokens
bcryptjs	Password hashing
Multer	File upload handling

Technology	Kaam
Cloudinary	Image/file cloud storage
dotenv	Environment variables
cors	Cross-origin requests
helmet	Security headers

5. Functional Requirements

5.1 Authentication Module

ID	Requirement	FE Kaam	BE Kaam	Priority
FR-01	User register ho sake (name, email, password)	Register form + validation	Register API + password hash	☐ High
FR-02	User login kar sake (email + password)	Login form + token store	Login API + JWT generate	☐ High
FR-03	Logout functionality	Token remove, redirect	Token blacklist (optional)	☐ High
FR-04	Password forgot / reset	Forgot password form	Reset email + token API	☐ Medium
FR-05	Profile photo upload on register	Image picker + preview	Upload API + Cloudinary	☐ Medium

5.2 Chat Module

ID	Requirement	FE Kaam	BE Kaam	Priority
FR-06	Chat list screen (sab conversations)	Chat list UI + real-time update	GET conversations API	☐ High
FR-07	One-to-one chat (text messages)	Chat bubbles UI + input	Message CRUD APIs + Socket	☐ High
FR-08	Real-time message delivery	Socket.IO client listen	Socket.IO server emit	☐ High
FR-09	Message status (sent ✓, delivered ✓✓, read □)	Status icons show	Status update logic	☐ Medium
FR-10	Typing indicator ("typing...")	Show typing text	Typing event broadcast	☐ Medium
FR-11	Message delete (for me / for everyone)	Delete UI + confirmation	Delete API + socket event	☐ Medium
FR-12	Emoji support	Emoji picker component	Emoji text store in DB	☐ High
FR-13	Message search	Search bar + results UI	Search API with query	☐ Low

5.3 Group Chat Module

ID	Requirement	FE Kaam	BE Kaam	Priority
----	-------------	---------	---------	----------

ID	Requirement	FE Kaam	BE Kaam	Priority
FR-14	Create group (name, photo, members)	Create group form	Group CRUD API	☐ High
FR-15	Group chat messaging	Same chat UI (group mode)	Group message broadcast	☐ High
FR-16	Add/remove group members	Member management UI	Member update API	☐ Medium
FR-17	Group admin controls	Admin badge + options	Admin permission check	☐ Medium

5.4 Media Sharing Module

ID	Requirement	FE Kaam	BE Kaam	Priority
FR-18	Image share in chat	Image picker + preview	Image upload + Cloudinary	☐ High
FR-19	File/document share	File picker	File upload API	☐ Medium
FR-20	Voice note (bonus)	Recorder UI	Audio upload API	☐ Low

5.5 Profile & Settings Module

ID	Requirement	FE Kaam	BE Kaam	Priority
FR-21	View own profile	Profile screen UI	GET profile API	☐ High
FR-22	Edit profile (name, bio, photo)	Edit form + image upload	Update profile API	☐ High
FR-23	View other user's profile	Profile screen (read-only)	GET user by ID API	☐ Medium
FR-24	Online/Offline status	Green dot show	Socket connect/disconnect track	☐ High
FR-25	Last seen timestamp	"Last seen at..." text	Last seen update in DB	☐ Medium

5.6 Call Module (UI Only — 5 Din Mein Functional Nahi)

ID	Requirement	FE Kaam	BE Kaam	Priority
FR-26	Call history list screen	UI screen only	Call log API (basic)	☐ Low
FR-27	Voice/Video call UI	Static call screen UI —		☐ Low

5.7 Status/Stories Module

ID	Requirement	FE Kaam	BE Kaam	Priority
FR-28	Post text/image status	Status form + preview	Status CRUD API	☐ Low
FR-29	View others' status	Status viewer UI	GET statuses API	☐ Low
FR-30	Status auto-delete after 24hrs	—	Cron job / TTL index	☐ Low

6. Non-Functional Requirements

6.1 Performance

Metric	Target
Message delivery	< 300ms (tezi se)
Page load time	< 2 seconds
API response	< 500ms
Image upload	< 3 seconds (5MB tak)

6.2 Security

Requirement	Implementation
Password encryption	bcryptjs (10 salt rounds)
Token authentication	JWT (24hr expiry)
API protection	Auth middleware on protected routes
Input validation	express-validator on backend
XSS prevention	Helmet.js + input sanitization
CORS	Whitelisted frontend URL only

6.3 Responsive Design

- **Desktop:** 1200px+ (main target)
- **Tablet:** 768px - 1199px
- **Mobile:** 320px - 767px

7. Database Design

7.1 Users Collection

```
// MongoDB - Users
{
  _id: ObjectId,
  fullName: String,           // "Kashif Ahmed"
  email: String (unique),     // "kashif@gmail.com"
  password: String (hashed),  // bcrypt hash
  profilePhoto: String,       // Cloudinary URL
  bio: String,                // "Hey there! I'm using CM Chat"
  phoneNumber: String,        // "+923001234567"
  isOnline: Boolean,          // true/false
  lastSeen: Date,             // last active timestamp
  createdAt: Date,
  updatedAt: Date
}
```

7.2 Conversations Collection

```
// MongoDB - Conversations
{
  _id: ObjectId,
  type: String,           // "private" | "group"
  participants: [ObjectId], // ref: Users (array of user IDs)

  // Group specific fields
  groupName: String,      // "Office Team" (sirf group ke liye)
  groupPhoto: String,     // Cloudinary URL
  groupAdmin: [ObjectId], // ref: Users

  lastMessage: {
    text: String,
    sender: ObjectId,
    timestamp: Date
  },

  createdAt: Date,
  updatedAt: Date
}
```

7.3 Messages Collection

```
// MongoDB - Messages
{
  _id: ObjectId,
  conversationId: ObjectId,    // ref: Conversations
  sender: ObjectId,           // ref: Users

  messageType: String,        // "text" | "image" | "file" | "audio"
  text: String,                // message content
  mediaUrl: String,            // Cloudinary URL (images/files)

  status: String,              // "sent" | "delivered" | "read"

  isDeleted: Boolean,          // soft delete
  deletedFor: [ObjectId],       // "delete for me" tracking

  readBy: [{
    user: ObjectId,
    readAt: Date
  }],

  createdAt: Date,
  updatedAt: Date
}
```

7.4 Statuses Collection (Optional — Day 5)

```
// MongoDB - Statuses
{
  _id: ObjectId,
  user: ObjectId,              // ref: Users
  type: String,                // "text" | "image"
  content: String,             // text ya image URL
  backgroundColor: String,     // text status ke liye bg color
  viewedBy: [ObjectId],        // kisne dekhi
  expiresAt: Date,             // 24hr auto-delete (TTL index)
  createdAt: Date
}
```

8. API Endpoints Contract

⚠ **BAHUT ZAROORI:** Yeh contract dono developers ko **Day 1** par agree karna hai. Frontend iske hisaab se mock data banayega, Backend iske hisaab se APIs banayega.

8.1 Authentication APIs

```
POST    /api/auth/register
        Body: { fullName, email, password, profilePhoto? }
        Response: { success, message, token, user }

POST    /api/auth/login
        Body: { email, password }
        Response: { success, token, user }

GET     /api/auth/me
        Headers: Authorization: Bearer <token>
        Response: { success, user }

POST    /api/auth/logout
        Headers: Authorization: Bearer <token>
        Response: { success, message }
```

8.2 User APIs

```
GET     /api/users/search?q=<query>
        → Search users by name/email
        Response: { success, users: [...] }

GET     /api/users/:userId
        → Get user profile
        Response: { success, user }

PUT     /api/users/profile
        Body: { fullName?, bio?, profilePhoto? }
        Response: { success, user }
```

8.3 Conversation APIs

```
GET    /api/conversations
      → Get all conversations of logged-in user
      Response: { success, conversations: [...] }

POST   /api/conversations
      Body: { participantId } (for private)
      Body: { participants: [...], groupName, groupPhoto } (for group)
      Response: { success, conversation }

GET    /api/conversations/:conversationId
      → Get single conversation with details
      Response: { success, conversation }
```

8.4 Message APIs

```
GET    /api/messages/:conversationId
      → Get all messages of a conversation
      Query: ?page=1&limit=50
      Response: { success, messages: [...], pagination }

POST   /api/messages
      Body: { conversationId, text?, mediaUrl?, messageType }
      Response: { success, message }

DELETE /api/messages/:messageId
      Query: ?type=forMe | ?type=forEveryone
      Response: { success, message }
```

8.5 Media Upload API

```
POST   /api/upload
      Body: FormData { file }
      Headers: Content-Type: multipart/form-data
      Response: { success, url, fileType, fileName }
```

9. WebSocket Events

Yeh dono developers ko samajhna zaroori hai — Frontend LISTEN karega, Backend EMIT karega

9.1 Connection Events

□ Client → Server:

"setup" → User connect hua (userId bhejo)
"disconnect" → User offline gaya

□ Server → Client:

"connected" → Connection confirm
"user-online" → Koi user online aaya { userId }
"user-offline" → Koi user offline gaya { userId }

9.2 Chat Events

□ Client → Server:

"join-chat" → Specific chat room join karo { conversationId }
"send-message" → Naya message bhejo { conversationId, text, sender }
"typing" → Main type kar raha hoon { conversationId, userId }
"stop-typing" → Typing band ki { conversationId, userId }
"message-read" → Message parh liya { conversationId, userId }

□ Server → Client:

"new-message" → Naya message aaya { message object }
"typing" → Koi type kar raha hai { userId }
"stop-typing" → Typing band ki { userId }
"message-delivered" → Message deliver ho gaya { messageId }
"message-read" → Message parh liya gaya { messageId, userId }

9.3 Group Events

```

□ Client → Server:
  "join-group"      → Group room join karo { conversationId }
  "send-group-message" → Group mein message bhejo

□ Server → Client:
  "new-group-message" → Group mein naya message aaya
  "member-added"      → Naya member add hua
  "member-removed"    → Member remove hua

```

10. 5-Din Ka Day-by-Day Plan

[!IMPORTANT] Har din shuru hone se pehle dono developers 15-minute standup call karein — kya kiya, kya karenge, koi blocker hai?

□ DAY 1 — Foundation (Bunyaad)

Goal: Project setup + Authentication + API contract finalize

□ Din Shuru Hone Par (Pehle 30 Min):

```

□ DONO MIL KAR:
├─ Figma design ko ek saath dekhna
├─ API contract (Section 8) par agree karna
├─ Git repository setup karna
├─ .env variables decide karna
└─ Communication channel set karna (Discord/Slack)

```

□ FRONTEND (Developer 1) — Day 1

Time	Kaam	Details
☀ Morning	Project Setup	Next.js 14 project create, Tailwind CSS configure, folder structure banao
☀ Morning	Folder Structure	components/, app/, hooks/, lib/, context/, utils/ banao
□ Mid-Morning	Auth Pages UI	Login screen Figma se code karo (email + password inputs, login button, register link)
□ Mid-Morning	Auth Pages UI	Register screen code karo (name, email, password, profile photo upload)
□ Afternoon	Auth Logic	React Hook Form se form validation lagao

Time	Kaam	Details
☐ Afternoon	Auth Context	AuthContext banao — login state, token storage (localStorage), logout function
☐ Evening	API Integration	Login + Register forms ko Backend APIs se connect karo (Axios)
☐ Evening	Protected Routes	Auth middleware — agar logged in nahi to login page par redirect

Day 1 Frontend Deliverables:

- ☐ Project initialized with Next.js + Tailwind
- ☐ Login screen (Figma match)
- ☐ Register screen (Figma match)
- ☐ Auth context with token management
- ☐ Login/Register API integration
- ☐ Protected route wrapper

☐ BACKEND (Developer 2) — Day 1

Time	Kaam	Details
☼ Morning	Project Setup	Node.js + Express project create, folder structure banao, packages install karo
☼ Morning	Database Setup	MongoDB Atlas par database banao, Mongoose connect karo
☐ Mid-Morning	User Model	User schema banao (Section 7.1 ke mutabiq)
☐ Mid-Morning	Auth APIs	POST <code>/api/auth/register</code> — validate, hash password, save user, return JWT
☐ Afternoon	Auth APIs	POST <code>/api/auth/login</code> — find user, compare password, return JWT
☐ Afternoon	Auth APIs	GET <code>/api/auth/me</code> — token verify karke user return karo
☐ Evening	Middleware	Auth middleware banao (JWT verify), Error handling middleware
☐ Evening	Cloudinary	Cloudinary setup for profile photo upload
☐ Evening	CORS + Helmet	Security setup, frontend URL whitelist

Day 1 Backend Deliverables:

- ☐ Express server running on port 5000
- ☐ MongoDB connected
- ☐ User model created
- ☐ Register API working (with password hashing)
- ☐ Login API working (with JWT)
- ☐ Auth middleware (token verification)
- ☐ Cloudinary setup for image upload
- ☐ CORS configured for frontend URL

☐ Day 1 End — Coordination Check:

- ☐ Frontend login form → Backend login API = WORKING
- ☐ Frontend register form → Backend register API = WORKING
- ☐ Token store in frontend → Token verify in backend = WORKING

☐ DAY 2 — Chat Core (Chat Ka Dil)

Goal: Chat list + One-to-one messaging + Real-time setup

☐ FRONTEND (Developer 1) — Day 2

Time	Kaam	Details
☀ Morning	Layout	Main app layout banao: Sidebar (chat list) + Main area (chat window)
☀ Morning	Chat List UI	Chat list component — user photo, name, last message, timestamp, unread badge
☐ Mid-Morning	Chat Window UI	Chat detail screen — header (user info), message bubbles (sent/received), input bar
☐ Mid-Morning	Message Bubbles	Sent bubble (right, blue) + Received bubble (left, dark) + timestamp below
☐ Afternoon	Socket.IO Client	Socket.IO client setup, connection establish karo with backend
☐ Afternoon	Real-time Messages	<code>send-message</code> emit karo, <code>new-message</code> listen karo
☐ Evening	Chat API Integration	GET conversations API se chat list load karo
☐ Evening	Messages Load	GET messages API se messages load karo jab chat open ho

Day 2 Frontend Deliverables:

- ☐ Chat list sidebar (Figma match)
- ☐ Chat window with message bubbles
- ☐ Message input bar with send button
- ☐ Socket.IO client connected
- ☐ Real-time message send/receive working
- ☐ Chat list loads from API
- ☐ Messages load when chat opens

☐ BACKEND (Developer 2) — Day 2

Time	Kaam	Details
✱ Morning	Models	Conversation model + Message model banao (Section 7.2, 7.3)
✱ Morning	Conversation APIs	GET /api/conversations — user ki sab conversations return karo
☐ Mid-Morning	Conversation APIs	POST /api/conversations — naya conversation create karo (private)
☐ Mid-Morning	Message APIs	GET /api/messages/:conversationId — messages return karo (paginated)
☐ Afternoon	Message APIs	POST /api/messages — naya message save karo
☐ Afternoon	Socket.IO Server	Socket.IO server setup, user connection handle karo
☐ Evening	Socket Events	"send-message" → message save + recipient ko emit
☐ Evening	Socket Events	"join-chat" → user ko specific room mein dalo
☐ Evening	Online Status	User connect/disconnect par online status update karo

Day 2 Backend Deliverables:

- ☐ Conversation model + Message model
- ☐ Conversation CRUD APIs
- ☐ Message CRUD APIs (with pagination)
- ☐ Socket.IO server running
- ☐ Real-time message broadcasting
- ☐ User online/offline tracking via socket
- ☐ Chat room (join/leave) functionality

☐ Day 2 End — Coordination Check:

- ☐ Frontend chat list → Backend GET conversations = WORKING
- ☐ Frontend send message → Socket → Backend save + broadcast = WORKING
- ☐ Frontend receive message → Real-time update = WORKING
- ☐ User online/offline → Green dot show = WORKING

□ DAY 3 — Features (Advanced Chat Features)

Goal: Typing indicator, message status, media sharing, user search, contacts

□ FRONTEND (Developer 1) — Day 3

Time	Kaam	Details
✱ Morning	Typing Indicator	"typing..." animation show karo jab doosra type kare
✱ Morning	Message Status	Tick marks dikhao — ✓ sent, ✓✓ delivered, □✓✓ read
□ Mid-Morning	Emoji Picker	Emoji picker component lagao (emoji-picker-react)
□ Mid-Morning	Image Send	Image select → preview → upload → send as message
□ Afternoon	User Search	Search bar → API call → results show → start new chat
□ Afternoon	Contacts Page	Contact list screen — sab registered users dikhao
□ Evening	New Chat Flow	User select → new conversation create → chat open
□ Evening	Message Delete	Long press / right click → delete option → confirmation

Day 3 Frontend Deliverables:

- Typing indicator working
- Message status ticks showing
- Emoji picker integrated
- Image sharing in chat
- User search functionality
- Contacts list page
- Start new chat flow
- Message delete (for me / for everyone)

□ BACKEND (Developer 2) — Day 3

Time	Kaam	Details
✱ Morning	Typing Socket	"typing" aur "stop-typing" events handle karo
✱ Morning	Message Status	Message status update logic — sent → delivered → read
□ Mid-Morning	File Upload API	POST /api/upload — Multer + Cloudinary integration
□ Mid-Morning	Media Messages	Image/file messages save with mediaUrl
□ Afternoon	User Search API	GET /api/users/search?q=query — name/email se search
□ Afternoon	Message Delete API	DELETE /api/messages/:id — soft delete + socket notify
□ Evening	Unread Count	Har conversation ka unread message count calculate karo
□ Evening	Last Message Update	Conversation mein lastMessage field update karo har naye message par

Day 3 Backend Deliverables:

- ☐ Typing events broadcast
- ☐ Message status tracking (sent/delivered/read)
- ☐ File upload API (Cloudinary)
- ☐ User search API
- ☐ Message delete API
- ☐ Unread message count
- ☐ Last message auto-update in conversation

☐ Day 3 End — Coordination Check:

- ☐ Typing indicator → real-time both sides = WORKING
- ☐ Image upload → Cloudinary → show in chat = WORKING
- ☐ User search → start new chat = WORKING
- ☐ Message delete → both sides update = WORKING
- ☐ Message status ticks → real-time update = WORKING

☐ DAY 4 — Groups + Profile + Polish

Goal: Group chat, profile screens, notifications, UI polishing

☐ FRONTEND (Developer 1) — Day 4

Time	Kaam	Details
✱ Morning	Create Group UI	Group name input, photo select, members select form
✱ Morning	Group Chat UI	Group chat screen — same as private but with member names on messages
☐ Mid-Morning	Group Info	Group info panel — members list, admin badge, leave group button
☐ Mid-Morning	My Profile	Profile screen — photo, name, bio, edit button
☐ Afternoon	Edit Profile	Edit profile form — name, bio, photo change
☐ Afternoon	Other User Profile	View other user's profile — photo, name, bio, shared media
☐ Evening	Notifications	Toast notifications for new messages (jab doosri chat open ho)
☐ Evening	UI Polish	Figma se match karo — spacing, colors, fonts, responsive

Day 4 Frontend Deliverables:

- ☐ Create group flow working
- ☐ Group chat messaging working
- ☐ Group info/details screen
- ☐ My profile screen
- ☐ Edit profile (with photo change)
- ☐ Other user profile view
- ☐ Toast notifications
- ☐ UI matches Figma design

☐ BACKEND (Developer 2) — Day 4

Time	Kaam	Details
☀ Morning	Group APIs	POST <code>/api/conversations</code> (group type) — group create
☀ Morning	Group APIs	PUT group — add member, remove member, change name/photo
☐ Mid-Morning	Group Socket	Group room management — broadcast to all group members
☐ Mid-Morning	Profile APIs	PUT <code>/api/users/profile</code> — update name, bio, photo
☐ Afternoon	Profile APIs	GET <code>/api/users/:id</code> — get other user profile
☐ Afternoon	Notification Logic	Track which user is in which chat room (for notification decisions)
☐ Evening	Bug Fixes	Sab APIs test karo, edge cases handle karo
☐ Evening	Validation	Input validation lagao sab APIs par (express-validator)

Day 4 Backend Deliverables:

- ☐ Group CRUD APIs complete
- ☐ Group socket broadcasting
- ☐ Profile update API
- ☐ User profile view API
- ☐ All APIs validated
- ☐ Edge cases handled
- ☐ Bug fixes done

☐ Day 4 End — Coordination Check:

- ☐ Create group → API → show in chat list = WORKING
- ☐ Group messaging → real-time all members = WORKING
- ☐ Profile edit → API → data update = WORKING
- ☐ Notifications → toast show on new message = WORKING

□ DAY 5 — Testing + Deployment + Final Touch

Goal: Full testing, bug fixes, deployment, documentation

□ FRONTEND (Developer 1) — Day 5

Time	Kaam	Details
☀ Morning	Call Screens	Call history screen (UI only), voice call screen (UI only)
☀ Morning	Status Screen	Status list screen (UI only — basic)
□ Mid-Morning	Responsive Test	Sab screens mobile + tablet par test karo
□ Mid-Morning	Bug Fixes	UI bugs fix karo — alignment, spacing, overflow issues
□ Afternoon	Settings Page	Basic settings page — logout, dark mode toggle (agar waqt ho)
□ Afternoon	Loading States	Sab screens par loading spinners aur skeleton screens lagao
□ Evening	Deployment	Vercel par frontend deploy karo
□ Evening	Testing	Full flow test: Register → Login → Chat → Group → Profile

Day 5 Frontend Deliverables:

- Call screens (UI only)
- Status screen (UI only)
- Responsive on all devices
- All bugs fixed
- Loading states added
- Deployed on Vercel
- Full flow tested

□ BACKEND (Developer 2) — Day 5

Time	Kaam	Details
☀ Morning	Status APIs	Basic status CRUD (optional — agar waqt ho)
☀ Morning	Call Log	Basic call log model + API (optional)
□ Mid-Morning	Security Review	Rate limiting, input sanitization, error messages check
□ Mid-Morning	Bug Fixes	API bugs fix karo — edge cases, error handling
□ Afternoon	Environment	Production .env setup, MongoDB Atlas production cluster
□ Afternoon	Deployment	Render/Railway par backend deploy karo
□ Evening	Frontend Connect	Deployed frontend ko deployed backend se connect karo
□ Evening	Final Testing	Postman + live app par full test karo

Day 5 Backend Deliverables:

- ☐ Status APIs (basic)
- ☐ Security hardened
- ☐ All bugs fixed
- ☐ Deployed on Render/Railway
- ☐ Production DB connected
- ☐ Frontend-Backend connected on production
- ☐ Full flow tested live

☐ Day 5 End — Final Coordination:

- ☐ Live URL working: Frontend (Vercel) → Backend (Render)
- ☐ Register → Login → Chat → Group → Profile = FULL FLOW WORKING
- ☐ Real-time messaging working on production
- ☐ Images uploading on production
- ☐ Mobile responsive working

11. Frontend-Backend Coordination Plan

11.1 Daily Standup (Har Roz 15 Min)

- ☐ Har din subah shuru hone par:
 - └─ ☐ Main ne kal kya kiya?
 - └─ ☐ Aaj kya karunga?
 - └─ ☐ Koi blocker hai?
 - └─ ☐ Koi API change chahiye?

11.2 Communication Rules

Rule	Details
API Changes	Agar koi API change kare → dono ko batana zaroori hai
Blocker	Agar kisi ko block ho → turant message karo, wait mat karo
Testing	Har API banne ke baad Frontend ko notify karo
Git	Har kaam ke baad commit karo — chhote chhote commits
End of Day	Roz end par 5 min demo do ek doosre ko

11.3 Coordination Matrix — Kaun Kab Kise Chahiye?

□ DEPENDENCY CHART:

Day 1:

BE: Auth APIs ready by afternoon → FE: Integrate by evening

Day 2:

BE: Socket server ready by mid-day → FE: Connect socket by afternoon

BE: Chat APIs ready by afternoon → FE: Load chats by evening

Day 3:

BE: Upload API ready by mid-day → FE: Image share by afternoon

BE: Search API ready by afternoon → FE: User search by evening

Day 4:

BE: Group APIs ready by mid-day → FE: Group features by evening

BE: Profile APIs ready by afternoon → FE: Profile screens by evening

Day 5:

BE: Deploy by afternoon → FE: Connect + Test by evening

12. Frontend-Backend Connection Guide

12.1 Environment Variables

Frontend (.env.local):

```
NEXT_PUBLIC_API_URL=http://localhost:5000/api
NEXT_PUBLIC_SOCKET_URL=http://localhost:5000
NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME=your_cloud_name
```

Backend (.env):

```
PORT=5000
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/cm-chat-app
JWT_SECRET=your_super_secret_key_here
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
FRONTEND_URL=http://localhost:3000
```

12.2 Axios Setup (Frontend)

```
// lib/axios.js
import axios from 'axios';

const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL,
  headers: {
    'Content-Type': 'application/json',
  },
});

// Har request mein token automatically lagao
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Agar 401 aaye to logout kar do
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

export default api;
```

12.3 Socket.IO Connection (Frontend)

```
// lib/socket.js
import { io } from 'socket.io-client';

const SOCKET_URL = process.env.NEXT_PUBLIC_SOCKET_URL;
let socket = null;

export const connectSocket = (userId) => {
  socket = io(SOCKET_URL, {
    query: { userId },
  });

  socket.on('connect', () => {
    console.log('Socket connected:', socket.id);
    socket.emit('setup', userId);
  });

  return socket;
};

export const getSocket = () => socket;

export const disconnectSocket = () => {
  if (socket) {
    socket.disconnect();
    socket = null;
  }
};
```

12.4 Socket.IO Server (Backend)

```
// socket/index.js
const { Server } = require('socket.io');

const initializeSocket = (server) => {
  const io = new Server(server, {
    cors: {
      origin: process.env.FRONTEND_URL,
      methods: ['GET', 'POST'],
    },
  });

  const onlineUsers = new Map(); // userId → socketId

  io.on('connection', (socket) => {
    const userId = socket.handshake.query.userId;

    // User online mark karo
    onlineUsers.set(userId, socket.id);
    io.emit('user-online', { userId });

    // Chat room join karo
    socket.on('join-chat', (conversationId) => {
      socket.join(conversationId);
    });

    // Naya message
    socket.on('send-message', (data) => {
      socket.to(data.conversationId).emit('new-message', data);
    });

    // Typing events
    socket.on('typing', (data) => {
      socket.to(data.conversationId).emit('typing', data);
    });

    socket.on('stop-typing', (data) => {
      socket.to(data.conversationId).emit('stop-typing', data);
    });

    // Disconnect
    socket.on('disconnect', () => {
      onlineUsers.delete(userId);
      io.emit('user-offline', { userId });
    });
  });
};
```

```
    });  
  });  
  
  return io;  
};  
  
module.exports = initializeSocket;
```

12.5 CORS Setup (Backend)

```
// server.js mein  
const cors = require('cors');  
  
app.use(cors({  
  origin: process.env.FRONTEND_URL, // "http://localhost:3000"  
  credentials: true,  
  methods: ['GET', 'POST', 'PUT', 'DELETE'],  
}));
```

12.6 Folder Structures

Frontend Folder Structure:

```
cm-chat-app-frontend/
├─ app/
|   ├─ (auth) /
|   |   ├─ login/page.jsx
|   |   └─ register/page.jsx
|   ├─ (main) /
|   |   ├─ chat/[conversationId]/page.jsx
|   |   ├─ contacts/page.jsx
|   |   ├─ groups/new/page.jsx
|   |   ├─ profile/page.jsx
|   |   ├─ profile/edit/page.jsx
|   |   ├─ calls/page.jsx
|   |   ├─ status/page.jsx
|   |   └─ settings/page.jsx
|   ├─ layout.jsx
|   ├─ page.jsx          ← redirects to /chat or /login
|   └─ globals.css
├─ components/
|   ├─ chat/
|   |   ├─ ChatList.jsx
|   |   ├─ ChatWindow.jsx
|   |   ├─ MessageBubble.jsx
|   |   ├─ MessageInput.jsx
|   |   └─ TypingIndicator.jsx
|   ├─ shared/
|   |   ├─ Avatar.jsx
|   |   ├─ SearchBar.jsx
|   |   ├─ LoadingSpinner.jsx
|   |   └─ Modal.jsx
|   ├─ layout/
|   |   ├─ Sidebar.jsx
|   |   └─ Header.jsx
|   └─ profile/
|       └─ ProfileCard.jsx
├─ context/
|   ├─ AuthContext.jsx
|   └─ ChatContext.jsx
├─ hooks/
|   ├─ useSocket.js
|   └─ useAuth.js
├─ lib/
|   ├─ axios.js
|   └─ socket.js
```

```
├─ utils/
|   └─ helpers.js
├─ public/
├─ .env.local
├─ package.json
└─ tailwind.config.js
```

Backend Folder Structure:

```
cm-chat-app-backend/
├─ config/
|   ├─ db.js           ← MongoDB connection
|   └─ cloudinary.js   ← Cloudinary config
├─ controllers/
|   ├─ authController.js
|   ├─ userController.js
|   ├─ conversationController.js
|   └─ messageController.js
├─ middleware/
|   ├─ authMiddleware.js ← JWT verify
|   ├─ errorMiddleware.js
|   └─ uploadMiddleware.js ← Multer config
├─ models/
|   ├─ User.js
|   ├─ Conversation.js
|   └─ Message.js
├─ routes/
|   ├─ authRoutes.js
|   ├─ userRoutes.js
|   ├─ conversationRoutes.js
|   ├─ messageRoutes.js
|   └─ uploadRoutes.js
├─ socket/
|   └─ index.js         ← Socket.IO setup
├─ utils/
|   └─ generateToken.js
├─ .env
├─ server.js           ← Entry point
└─ package.json
```

13. Testing Checklist

13.1 Authentication Tests

- ☐ ☐ User register ho sake (valid data)
- ☐ ☐ Duplicate email par error aaye
- ☐ ☐ Wrong password par error aaye
- ☐ ☐ Login ke baad token mile
- ☐ ☐ Protected routes bina token ke block hon
- ☐ ☐ Logout ke baad redirect ho

13.2 Chat Tests

- ☐ ☐ Chat list load ho (with last message)
- ☐ ☐ Messages load hon jab chat open ho
- ☐ ☐ Text message bhej sake — real-time dikhaye
- ☐ ☐ Image bhej sake — upload + show
- ☐ ☐ Typing indicator dikhaye
- ☐ ☐ Message status ticks update hon
- ☐ ☐ Message delete ho sake

13.3 Group Tests

- ☐ ☐ Group create ho sake
- ☐ ☐ Group mein message sab ko mile
- ☐ ☐ Members add/remove ho sakein
- ☐ ☐ Group info dikhaye

13.4 Profile Tests

- ☐ ☐ Profile dikhaye (photo, name, bio)
- ☐ ☐ Profile edit ho sake
- ☐ ☐ Doosre ka profile dekh sake

13.5 Responsive Tests

- ☐ ☐ Desktop (1200px+) — sab theek dikhaye
- ☐ ☐ Tablet (768px) — layout adjust ho
- ☐ ☐ Mobile (375px) — stack layout, hamburger menu

14. Risk Analysis

Risk	Probability	Impact	Solution
Socket connection drop	☐ Medium	☐ High	Auto-reconnect logic lagao

Risk	Probability	Impact	Solution
API slow response	☐ Medium	☐ Medium	Loading states + pagination
Image upload fail	☐ Medium	☐ Medium	Retry logic + error toast
CORS issues	☐ High	☐ Medium	Day 1 par CORS properly configure karo
MongoDB free tier limit	☐ Low	☐ High	Data cleanup + monitoring
Time exceed (5 din)	☐ High	☐ High	Low priority items skip karo (calls, status)
Merge conflicts (Git)	☐ Medium	☐ Medium	Separate folders, frequent commits

☐ Final Notes

Priority Order (Agar Waqt Kam Ho):

☐ MUST HAVE (Zaroor karo):

- └─ Authentication (Login/Register)
- └─ Chat List
- └─ One-to-one messaging (real-time)
- └─ Image sharing
- └─ User search + New chat
- └─ Profile view/edit
- └─ Deployment

☐ SHOULD HAVE (Koshish karo):

- └─ Group chat
- └─ Typing indicator
- └─ Message status ticks
- └─ Message delete
- └─ Responsive design

☐ NICE TO HAVE (Agar waqt bache):

- └─ Call screens (UI only)
- └─ Status/Stories
- └─ Voice notes
- └─ Dark mode toggle
- └─ Emoji reactions

Document Version: 1.0

Date: 23 September 2026

Prepared For: CM Chat App Development Team (2 Developers)

Figma Design: CM Chat App (<https://www.figma.com/design/0ow1aTjRRqjz2PjPeAn1f/>).

Status: Ready for Execution ☐

